

DATA STRUCTURES VIVA

Important Questions & Short, Exam-Ready Answers

Quick memory map: Stack → LIFO | Queue → FIFO | BFS → Queue | DFS → Stack/Recursion | BST Inorder → Sorted
| MST → $V-1$ edges | Dijkstra → SSSP | Floyd-Warshall → APSP

1. Data Structures

Q. What is a data structure?

Answer: A way of organizing and storing data so it can be accessed and processed efficiently.

Q. Why are data structures used?

Answer: For efficient storage, searching, insertion, deletion, and processing.

Q. Give examples.

Answer: Array, Stack, Queue, Linked List, Tree, Graph, Hash Table.

Q. What is ADT?

Answer: Abstract Data Type. It specifies operations without specifying implementation. Example: Stack has push, pop, and peek.

2. Classification

Q. How are data structures classified?

Answer: Mainly into Linear and Non-linear data structures.

Q. What are linear data structures?

Answer: Elements are arranged sequentially. Examples: Array, Stack, Queue, Linked List.

Q. What are non-linear data structures?

Answer: Elements are arranged hierarchically or as networks. Examples: Tree and Graph.

Q. Static vs dynamic?

Answer: Static size is fixed during execution, e.g. array. Dynamic size can change, e.g. linked list.

3. Stack

Q. What is a stack?

Answer: A linear data structure following LIFO — Last In, First Out.

Q. Basic operations?

Answer: push() inserts, pop() deletes the top element, peek()/top() views the top.

Q. Stack overflow?

Answer: Trying to insert into a full stack.

Q. Stack underflow?

Answer: Trying to delete from an empty stack.

Q. Applications?

Answer: Recursion, function calls, undo/redo, expression evaluation, parentheses matching, DFS.

Q. Push/pop complexity?

Answer: $O(1)$.

4. Queue

Q. What is a queue?

Answer: A linear data structure following FIFO — First In, First Out.

Q. Basic operations?

Answer: enqueue() inserts, dequeue() deletes, front() gives the first element, rear() gives the last.

Q. Queue overflow/underflow?

Answer: Overflow means inserting into a full queue; underflow means deleting from an empty queue.

Q. Applications?

Answer: CPU scheduling, printer queues, BFS, network buffering, task scheduling.

Q. Stack vs queue?

Answer: Stack uses LIFO; queue uses FIFO. Stack inserts/deletes at top; queue inserts at rear and deletes at front.

5. Linked List

Q. What is a linked list?

Answer: A dynamic linear structure made of nodes containing data and a link/pointer.

Q. What does a node contain?

Answer: Data and a pointer/link.

Q. Types?

Answer: Singly, Doubly, and Circular linked lists.

Q. Advantage over array?

Answer: It can grow or shrink dynamically.

Q. Disadvantages?

Answer: Extra pointer memory and no direct/random access.

Q. First node?

Answer: Head.

Q. Last node points to?

Answer: NULL in a normal singly linked list.

Q. Search complexity?

Answer: $O(n)$.

6. Trees – BST & Traversal

Q. What is a tree?

Answer: A non-linear hierarchical data structure of nodes connected by edges.

Q. What is a binary tree?

Answer: A tree where each node has at most two children.

Q. What is a BST?

Answer: A binary tree where left-subtree values are smaller than the root and right-subtree values are larger.

Q. What are traversals?

Answer: Inorder, Preorder, Postorder, and Level Order.

Q. Inorder?

Answer: Left → Root → Right (LNR).

Q. Preorder?

Answer: Root → Left → Right (NLR).

Q. Postorder?

Answer: Left → Right → Root (LRN).

Q. Special property of BST inorder?

Answer: It produces the elements in sorted order.

7. AVL, Red-Black & B-Tree

Q. What is an AVL tree?

Answer: A self-balancing Binary Search Tree.

Q. AVL balance factor?

Answer: Height(Left) – Height(Right), and it must be -1, 0, or +1.

Q. Why rotations?

Answer: To restore AVL balance. Types: LL, RR, LR, RL.

Q. What is a Red-Black Tree?

Answer: A self-balancing BST whose nodes are colored red or black and satisfy balancing rules.

Q. Why use Red-Black Trees?

Answer: They maintain approximately balanced height, giving $O(\log n)$ operations.

Q. What is a B-Tree?

Answer: A balanced multiway search tree; a node can contain multiple keys and children.

Q. Where are B-Trees used?

Answer: Databases, file systems, and disk-based storage.

Q. Why useful in databases?

Answer: They reduce disk accesses by storing many keys in each node.

8. Sorting – Quick, Merge, Bubble

Q. What is Bubble Sort?

Answer: Repeatedly compare adjacent elements and swap them if they are in the wrong order.

Q. Bubble Sort complexity?

Answer: Best $O(n)$ with optimization; average and worst $O(n^2)$.

Q. What is Quick Sort?

Answer: A divide-and-conquer algorithm that selects a pivot and partitions the array.

Q. Quick Sort complexity?

Answer: Average $O(n \log n)$, worst $O(n^2)$.

Q. What is Merge Sort?

Answer: Divide the array into smaller parts, sort them, and merge them.

Q. Merge Sort complexity?

Answer: Best, average, and worst $O(n \log n)$.

Q. Which use divide and conquer?

Answer: Quick Sort and Merge Sort.

9. Hashing

Q. What is hashing?

Answer: Mapping a key to a location in a hash table using a hash function.

Q. What is a hash function?

Answer: A function that converts a key into an index, e.g. $\text{index} = \text{key} \% \text{table_size}$.

Q. What is collision?

Answer: When two different keys produce the same hash index.

Q. Collision handling?

Answer: Chaining and open addressing; open addressing includes linear probing, quadratic probing, and double hashing.

Q. Average search time?

Answer: Approximately $O(1)$; worst case can be $O(n)$.

10. Graphs – BFS & DFS

Q. What is a graph?

Answer: A collection of vertices and edges.

Q. What is BFS?

Answer: Breadth-First Search visits vertices level by level.

Q. BFS uses which structure?

Answer: Queue.

Q. What is DFS?

Answer: Depth-First Search explores deeply along a path before backtracking.

Q. DFS uses which structure?

Answer: Stack; it can also be implemented with recursion.

Q. BFS applications?

Answer: Shortest path in an unweighted graph, level-order traversal, network traversal.

Q. DFS applications?

Answer: Cycle detection, path finding, topological sorting, connected components.

11. MST – Prim's & Kruskal's

Q. What is a spanning tree?

Answer: A tree that connects all vertices of a connected graph without cycles.

Q. What is MST?

Answer: A spanning tree with minimum possible total edge weight.

Q. MST edges for V vertices?

Answer: $V - 1$.

Q. What is Prim's algorithm?

Answer: Starts from a vertex and repeatedly chooses the minimum-weight edge connecting the current tree to a new vertex.

Q. What is Kruskal's algorithm?

Answer: Sorts edges by weight and adds the smallest edge that does not form a cycle.

Q. What does Kruskal commonly use?

Answer: Disjoint Set Union (DSU) / Union-Find.

Q. Prim vs Kruskal?

Answer: Prim grows a tree from a starting vertex; Kruskal processes globally sorted edges and joins components.

12. Shortest Paths – Dijkstra & Floyd-Warshall

Q. What is Dijkstra's algorithm?

Answer: Finds shortest paths from one source to all other vertices: Single Source Shortest Path (SSSP).

Q. Can Dijkstra use negative edges?

Answer: No. It requires non-negative edge weights.

Q. What is relaxation?

Answer: Checking whether a shorter path to a vertex can be obtained through another vertex.

Q. What is Floyd-Warshall?

Answer: Finds shortest paths between every pair of vertices: All-Pairs Shortest Path (APSP).

Q. What technique does Floyd-Warshall use?

Answer: Dynamic Programming.

Q. Floyd-Warshall complexity?

Answer: $O(V^3)$.

Q. Can Floyd-Warshall handle negative edges?

Answer: Yes, if there is no negative-weight cycle.

■ 20 ONE-LINE VIVA ANSWERS

Stack principle: LIFO — Last In, First Out.

Queue principle: FIFO — First In, First Out.

Stack insertion: Push.

Stack deletion: Pop.

Queue insertion: Enqueue.

Queue deletion: Dequeue.

BST property: Left < Root < Right.

BST inorder: Sorted order.

AVL balance factor: -1, 0, +1.

BFS: Queue.

DFS: Stack/Recursion.

MST edges: $V - 1$.

Kruskal: Union-Find / DSU.

Prim: Grows MST by minimum connecting edge.

Dijkstra: Single Source Shortest Path.

Dijkstra negative edges: No.

Floyd-Warshall: All-Pairs Shortest Path.

Floyd-Warshall complexity: $O(V^3)$.

Merge Sort: $O(n \log n)$.

Hashing average search: $O(1)$.